Project Planning & Management

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 16 October 2026

1. Project Proposal

1.1 Overview

An e-commerce company gets its data from different places: product data, customer data, order data and transaction data. This project builds an automated data pipeline that collects this data, cleans it, stores it in a relational database, and prepares a data model that is ready for analysis.

1.2 Problem Statement

The company receives data from CSV files, an external API and a product website. The data has duplicates, missing values, inconsistent formats and records that change over time. Analysts need one reliable central dataset to build sales reports.

1.3 Objectives

- Collect data from many sources: CSV/JSON files and a REST API.
- Check, clean and standardize the data (missing values, duplicates, data types and formats).
- Store the data in a relational database with a star schema.
- Run the whole process automatically with Apache Airflow.
- Make the data ready for sales reporting using SQL views.

1.4 Scope

What the project includes:

- Get product data from a REST API.
- Read customer and order data from CSV/JSON files.
- Validate the incoming data.
- Handle missing values and duplicates.
- Standardize data types and formats.
- Join and transform the data with Pandas.
- Design a relational database.
- Create a star schema: Fact Sales, Dim Customer, Dim Product and Dim Date.
- Load the data incrementally (append-based).
- Create SQL views for common analysis questions.
- Add indexes to the fields that are queried often.
- Build an Airflow DAG: Extract, Validate, Transform, Load, Quality Check.
- Add logging, retry and error handling.

1.5 Expected Deliverables

All deliverables are uploaded to one GitHub repository:

- Python ETL modules
- Database schema and ERD
- SQL scripts
- Airflow DAG
- Sample input datasets
- Data-quality checks
- Pipeline documentation
- Architecture diagram
- Screenshots of successful Airflow runs
- README file that explains the full architecture

2. Project Plan

2.1 Timeline (Gantt Chart)

The work is split into phases. A colored cell means the phase is active in that week. The weeks start on Monday.

Phase	Dates	5 Oct	12 Oct	19 Oct	26 Oct	2 Nov	9 Nov	16 Nov	23 Nov	30 Nov
Reading and summarizing the project	7 Oct									
Planning, Literature Review, Requirements	7 - 16 Oct									
System Analysis & Design	17 Oct - 6 Nov									
Implementation (build the pipeline)	7 - 30 Nov									
Final Presentation, Testing & Reports	1 - 4 Dec									

Note: the phase dates inside each deadline are our own plan. The official deadlines are in the milestones table below.

2.2 Milestones and Deliverables

Milestone	Deadline	Deliverable
M1 - Planning	16 Oct 2026	Project Planning & Management document, Literature Review, Requirements Gathering
M2 - Design	6 Nov 2026	System Analysis & Design: architecture, ERD, database schema, data flow diagrams
M3 - Implementation	30 Nov 2026	Working pipeline: source code, Airflow DAG, database, README, GitHub repository
M4 - Final	4 Dec 2026	Testing, reports, user manual, final presentation and infographic

2.3 Resource Allocation

Team: 6 members (see Section 3). The tools we use are:

Resource	Use in the project
Python 3.11	Writing the ETL code
Pandas	Cleaning, joining and transforming data
requests library	Getting product data from the REST API
SQLite	Relational database for the star schema
Apache Airflow (on WSL Ubuntu)	Running the pipeline automatically
GitHub	Storing code and all documents

3. Task Assignment & Roles

Each member is responsible for one stage of the pipeline.

#	Name	Role	Responsibility
1	نوران حمدي شوقي راغب	Database and Load	Designs the tables and star schema and loads the data into the database
2	حسانين رمضان محمد صبره	Validate	Checks the incoming data (missing values, data types) before processing
3	عبدالرحمن محمد أحمد صبري	SQL Views, Indexes and Quality Check	Creates the analytical views and indexes and checks data quality after loading
4	هدي نصر اسماعيل صالح	Transform	Cleans the data, removes duplicates and standardizes formats using Pandas
5	احمد محمد ربيع عبد الغفور	Airflow DAG, Logging and Retry	Connects all pipeline stages and schedules them to run automatically
6	عامر محمد صبري صابر احمد	Extract	Reads the CSV/JSON files and gets the products from the API

4. Risk Assessment & Mitigation Plan

Risk	Level	How we reduce it
Airflow does not run directly on Windows	High	Run Airflow inside WSL Ubuntu, and test the setup early.
The external API is down or changes its response	Medium	Keep a saved copy of a sample API response to test with; add retry for API calls.
Bad data breaks the pipeline (missing values, wrong types)	High	Validate the data first, handle errors, and run a Quality Check at the end.
A task fails in the middle of a run	Medium	Use Airflow retry and write clear logs to find the problem fast.
Team members have different skill levels	Medium	Give simple roles, help each other, and review the work together.
Missing a deadline	Medium	Follow the milestones and check progress every week.
Conflicts when many people edit the same files on GitHub	Low	Each member works on separate files, and we upload often.

5. KPIs (Key Performance Indicators)

KPI	Target
Successful pipeline runs in Airflow	All 5 tasks finish green (Extract, Validate, Transform, Load, Quality Check)
Duplicate records in the final tables	0
Missing values in key columns after cleaning	0
Valid records loaded into the database	100% of the records that passed validation
Data-quality checks passed	100%
Failed tasks that are retried and logged	Every failed task is retried and recorded in the log
Deliverables submitted on time	All 4 milestones by their deadlines

Literature Review

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 16 October 2026

1. Purpose

This document reviews the main ideas and tools related to our project. It explains what each idea means in simple words, and how we use it in the project. The ideas come from the DEPI Data Engineering course.

2. Data Integration and ETL

Data integration means combining data from many sources so the business can understand it and take decisions. It is different from data ingestion: ingestion only moves the data into the first storage layer, while integration also cleans and transforms it.

ETL stands for Extract, Transform, Load:

- Extract: get the data from the sources (files, APIs, websites).
- Transform: clean the data and put it in one common format.
- Load: save the clean data in the target database.

ELT does the same steps but loads the data first and transforms it after loading. We use ETL, because we clean the data with Pandas before we store it in the database.

Data can be loaded in full every time, or incrementally (only the new records). Our project uses incremental (append) loading.

3. Data Quality: Validation, Missing Values and Duplicates

Real data is rarely clean. The common problems are:

- Missing values (empty fields).
- Duplicate records (the same row more than once).
- Wrong or mixed data types and formats (for example, different date formats).

To handle them, the pipeline first validates the data, then cleans it, and at the end runs a quality check to make sure the final data is correct.

4. Data Modeling: Star Schema

A star schema is a way to design a data warehouse for analysis. It has one central fact table that stores the measurable events (like sales), connected to dimension tables that describe them (like customer, product and date). A snowflake schema is similar, but the dimension tables are split into smaller tables.

We use a star schema because it is simple and makes the analysis queries easy. Our model has: Fact Sales, Dim Customer, Dim Product and Dim Date.

5. Relational Databases, SQL Views and Indexes

- A relational database stores data in tables that are linked by keys.
- A view is a saved SQL query that analysts can use like a table.
- An index makes searching in a column faster.

We store our data in SQLite, create views for the common analysis questions, and add indexes on the fields that are queried often.

6. Pandas for Data Transformation

Pandas is a Python library for working with tables of data (DataFrames). It can read CSV and JSON files, remove duplicates, fill or drop missing values, change data types and join tables. We use it for all the transformation work.

7. Getting Data from a REST API

An API lets one program ask another program for data. The program sends a request, checks the response status code, and reads the JSON data if the request succeeded. We use the Python requests library to get the product data from an API.

8. Workflow Orchestration with Apache Airflow

Airflow runs and schedules workflows. A workflow is written as a DAG (Directed Acyclic Graph): a set of tasks with a clear order and no loops. Airflow can also retry failed tasks and keep logs of every run. Our DAG has five tasks: Extract, Validate, Transform, Load and Quality Check.

9. Choices We Compared

Topic	Options	Our choice and why
Product data source	REST API or web scraping	REST API. It is simpler and more stable, because scraping can break when the website changes.
Database	SQLite, MySQL or PostgreSQL	SQLite. It is a single file, needs no server and supports tables, views and indexes.
Transformation	Pandas or plain Python	Pandas. It is made for cleaning and joining tables.
Orchestration	Apache Airflow	Airflow. It is required by the project and it was taught in the course.

10. Summary

These ideas build the full pipeline: we extract data from files and an API, validate and clean it with Pandas, load it into a star schema in SQLite, create SQL views and indexes for analysis, run a quality check, and use Airflow to run everything automatically with logging and retry.

11. References

Course materials of the DEPI Data Engineering track:

- Module 1: Introduction to Data Engineering - Overview of Data Engineering
- Module 2: Python Programming - Pandas for Data Manipulation, Advanced Pandas
- Module 3: SQL & Database Management - Advanced SQL Techniques, Data Modeling
- Module 4: Advanced Python - Python for Databases & APIs
- Module 5: Data Pipelines - Data Integration & ETL Fundamentals, Data Pipeline Development with Airflow

Requirements Gathering

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 16 October 2026

1. Stakeholder Analysis

A stakeholder is any person or group that uses the system or is affected by it. These are the stakeholders of our pipeline and what each one needs:

Stakeholder	What they need	How they use the project
Data analysts	One reliable, clean and central dataset with no duplicates or missing values	Query the star schema and the SQL views to build sales reports
Business managers	Correct sales numbers to take decisions	Read the reports that come from the analysts' queries
Data engineering team (us)	A pipeline that runs automatically, is easy to understand and shows clear logs when something fails	Build, run and maintain the pipeline in Airflow
E-commerce company	Data from many sources combined in one place	Owns the data sources (files, API) and the final database
DEPI instructor and evaluators	All required deliverables, documented and uploaded to GitHub	Review and grade the project

2. User Stories and Use Cases

2.1 User Stories

ID	As a...	I want to...	So that...
US-1	Data analyst	get customer, order and product data in one database	I do not collect the data by hand from different sources
US-2	Data analyst	have duplicates and missing values handled	my reports are correct
US-3	Data analyst	use ready SQL views	I can answer common sales questions quickly
US-4	Business manager	see correct sales figures	I can take good decisions
US-5	Data engineer	run the whole pipeline automatically in Airflow	I do not run each step manually
US-6	Data engineer	see logs and retries when a task fails	I can find and fix the problem fast
US-7	Data engineer	have a quality check at the end of each run	I know the loaded data is correct
US-8	Instructor	find the code, documents and screenshots on GitHub	I can evaluate the project

2.2 Use Cases

Use case	Actor	Main steps	Result
UC-1: Run the pipeline	Data engineer	1. Open Airflow. 2. Start (trigger) the DAG. 3. Airflow runs Extract, Validate, Transform, Load and Quality Check.	Clean data is loaded into the database
UC-2: Check a pipeline run	Data engineer	1. Open the DAG run in Airflow. 2. See the status of each task. 3. Open the log of a failed task.	The engineer knows if the run succeeded or why it failed
UC-3: Analyze sales	Data analyst	1. Open the database. 2. Run a SQL view or query on the star schema.	Sales results for reports
UC-4: Review data quality	Data engineer	1. Open the Quality Check task result. 2. Read the checks that passed or failed.	Confirmation that the final data is correct

3. Functional Requirements

These are the features the system must have:

ID	Requirement
FR-1	The system shall get product data from a REST API.
FR-2	The system shall read customer and order data from CSV/JSON files.
FR-3	The system shall validate the incoming data (for example, required fields and data types).
FR-4	The system shall handle missing values.
FR-5	The system shall remove duplicate records.
FR-6	The system shall standardize data types and formats.
FR-7	The system shall join and transform the data using Pandas.
FR-8	The system shall store the data in a relational database with a star schema: Fact Sales, Dim Customer, Dim Product and Dim Date.
FR-9	The system shall load the data incrementally (append-based).
FR-10	The system shall provide SQL views for common analysis queries.
FR-11	The system shall have indexes on the fields that are queried often.
FR-12	The system shall run a data-quality check after loading.
FR-13	The system shall run through an Airflow DAG with the tasks: Extract, Validate, Transform, Load, Quality Check.
FR-14	The system shall write logs and retry a failed task.

4. Non-functional Requirements

These describe how well the system should work:

ID	Type	Requirement
NFR-1	Reliability	A bad record or a failed task must not stop the whole pipeline without a clear error; failed tasks are retried.
NFR-2	Data quality	The final tables must have no duplicates and no missing values in the key columns.
NFR-3	Performance	A full run on the sample data finishes in less than 1 minute (measured about 30 seconds in Airflow).
NFR-4	Usability	The README explains how to install and run the project step by step in simple words.
NFR-5	Maintainability	The code is split into small modules (extract, validate, transform, load, quality check) with clear names and comments.
NFR-6	Observability	Every step writes clear logs, and the run status is visible in the Airflow interface.
NFR-7	Portability	The project runs with Python 3.11 and Airflow on Ubuntu (WSL), using only free tools.
NFR-8	Security	No passwords or secret keys are written inside the code.